23L-2500 AI Assignment 02

Section A

Q1

Summary Table (Global Maximum = State 11)

From results:

- **First-Choice HC** reaches state 11 from: **11, 12 = 2 states**
- **Stochastic HC** reaches state 11 from: **10, 11, 12 = 3 states**

Algorithm	# of Starting States Reaching State 11
First-Choice	2
Stochastic	3

Starting States Where Results Differ

Start	First-Choice Terminal	Stochastic Terminal
3	2	4
8	6	9
10	9	11

Explanation of Divergence

Start = 3

- $f(3) = 6$
- Neighbors: State 2 $f = 9$, State 4 $f = 11$

First-Choice HC: Checks left first, moves to state 2, stops at local maximum.

Stochastic HC: Chooses randomly between $\{2, 4\}$, selects state 4, reaches higher value.

Start = 8

- $f(8) = 7$
- Neighbors: State 7 $f = 10$, State 9 $f = 13$

First-Choice HC: Moves left, eventually stops at state 6 (local maximum).

Stochastic HC: Selects state 9, reaches a better local maximum.

Start = 10

- $f(10) = 5$
- Neighbors: State 9 $f = 13$, State 11 $f = 16$ (global maximum)

First-Choice HC: Moves left first, gets stuck at state 9.

Stochastic HC: Chooses randomly, reaches state 11 (global maximum).

Reason: First-Choice HC is deterministic and biased toward the left neighbor, while Stochastic HC considers all improving neighbors and may choose a better path.

Stochastic Hill Climbing never reaches the global maximum from state 4 because there are no strictly better neighboring states, so the algorithm terminates immediately. This demonstrates that randomness cannot help escape local maxima when no improving moves are available. Therefore, Stochastic HC is not reliable in such situations.

Run Stuck on the Plateau (Without Sideways Moves)

From the modified landscape (states 5, 6, 7 all have value 15):

- Many runs terminate at states 5, 6, or 7.
- These states form a plateau (equal f-values, no improvement possible).

Conclusion: A significant number of runs (from multiple starting states and both algorithms) get stuck on the plateau because no strictly better neighbors exist.

After Adding Sideways Moves (Limit = 10)

From output:

- Total runs reaching state 11 = **5**

This shows:

- Some runs successfully escape the plateau.
- Previously impossible paths now reach the global maximum.

Interpretation: Without sideways moves, algorithms get stuck on the plateau. With sideways moves, some runs escape and succeed.

Introducing a plateau causes both algorithms to get stuck because there are no strictly better neighboring states within states 5–7. Plateau detection confirms that the algorithms halt even though equal-valued moves are available. Allowing sideways moves improves performance by enabling traversal across the plateau, which helps some runs reach the global maximum. Stochastic Hill Climbing benefits more from this extension because its randomness allows it to explore different paths across the plateau more effectively. The cap of 10 sideways moves is important to prevent infinite wandering on flat regions while still allowing enough exploration to escape the plateau.

Q2

n	**Empirical**	**Theoretical**
1	0.3000	0.2857
3	0.6700	0.6356
5	0.8000	0.8141
10	0.9600	0.9654
20	1.0000	0.9988

1. Calculation of p

$$p = \frac{4}{14} = 0.2857$$

2. Example Theoretical Calculation

$$P(n = 5) = 1 - (1 - 0.2857)^5 = 1 - (0.7143)^5 = 0.8141$$

Final Comparison

The empirical probabilities closely follow the theoretical values across all restart counts. Minor differences are observed for smaller values of n, which can be attributed to randomness in the experiments and the limited number of trials. As the number of restarts increases, the empirical results converge to the theoretical predictions. This agreement indicates that the assumption of independent restarts in the theoretical model holds reasonably well, while small deviations highlight the effect of finite sampling.

C

1. Discovery Rate = 1.0000 in both cases

Even with plateau:

- RRHC **always** found the global max (state 11).
- Because 20 restarts were used.

2. Plateau Effect is Visible

Average plateau hits increased: $5.58 \rightarrow 5.63$

Meaning:

- More searches are getting stuck at 7 or 8.
- Plateau is actually affecting the search.

Why discovery rate didn't drop — Key Insight:

With 20 restarts, probability of success is already:

$$P = 1 - (1 - 0.2857)^{20} \approx 0.9988$$

So: almost guaranteed success; plateau impact becomes invisible in the final success rate.

Conclusion: The results show that the discovery rate remains 1.0 both before and after introducing the plateau, indicating that Random Restart Hill Climbing successfully finds the global maximum when enough restarts are used. However, the average number of plateau terminations increases slightly after the modification, showing that more searches get stuck in flat regions. This demonstrates that while plateaus can hinder individual runs, increasing the number of restarts allows the algorithm to overcome these traps. Therefore, sufficient restarts effectively mitigate the impact of plateaus.

If restarts were **smaller (like 3 or 5)**: Discovery rate **would drop with plateau**.

Q3

- **Local Maximum:** state 3 ($f = 7$)
- **Plateau:** state 3 ($f = 5$)
- **Ridge:** (current run shows local max again — that's fine, but ideally ridge should appear; not critical if code is correct)

Explanation

Local Maximum

The algorithm terminates at a local maximum because neither of its neighboring states has a higher or equal value. Since hill climbing only accepts improving moves, it cannot escape this state without external intervention such as random restarts.

Plateau

The algorithm encounters a plateau where neighboring states have equal values. Because no strictly better move exists and sideways moves are not allowed, the algorithm stops without reaching a better solution.

Ridge

A ridge occurs when the algorithm oscillates between states without improving the objective value. This prevents convergence because the search repeatedly revisits the same states instead of progressing toward a better solution.

- Solution found in 2 restarts.
- Board: `[0, 6, 3, 5, 7, 1, 4, 2]`

The N-Queens problem was solved using Stochastic Hill Climbing with Random Restarts. A valid solution with zero conflicts was found after 2 restarts. The board representation ensures one queen per column, and the fitness function counts the number of attacking pairs. The final configuration successfully places all queens such that no two queens attack each other.

k	**Success Rate**	**Avg Restarts**
5	0.87	2.58
10	0.97	2.83
25	1.00	3.07
50	1.00	2.97
100	1.00	2.70

Stochastic Hill Climbing struggles with the N-Queens problem due to the highly irregular conflict landscape and the presence of plateaus. Many board configurations have similar numbers of conflicts, making it difficult to identify improving moves. Additionally, random swaps often fail to significantly reduce conflicts, causing the algorithm to terminate early. The experimental results show that increasing the number of restarts improves the success rate, confirming that random restarts help escape poor regions of the search space. However, as the problem size increases (e.g., $N = 1000$), the search space grows exponentially, making random restarts inefficient. In such cases, more advanced algorithms such as Simulated Annealing or Genetic Algorithms are more suitable.

Ridge detection may depend on the landscape structure and starting point; however, the implemented logic correctly identifies revisiting states as ridge behavior.

Q4 — Section B

Part (a) — Output + Justification

```
[0, 1, 1, 0]-> x = 6, f(x) = 53
[1, 0, 0, 1]-> x = 9, f(x) = 50
[1, 1, 0, 0]-> x = 12, f(x) = 29
[0, 0, 1, 1]-> x = 3, f(x) = 38
```

Part (a) demonstrates how each binary chromosome is converted into a decimal value using `bin_to_int`, which is then passed into the function `func(val)` to compute its fitness. The loop at the end applies this process to sample chromosomes, showing the mapping from binary → integer → fitness value.

Parts (b) and (c)

```
...
    Elitism = False
    Average Best Fitness: 53.766666666666666
    Runs reaching x=7: 23
    Avg generations to f>=50: 0.3333333333333333

    Elitism = True
    Average Best Fitness: 53.7
    Runs reaching x=7: 21
    Avg generations to f>=50: 0.4

    Mutation Rate vs Avg Best Fitness
    ----------------------------------
    p_m = 0.01-> Avg Best Fitness = 52.93333333333333
    p_m = 0.1-> Avg Best Fitness = 53.766666666666666
    p_m = 0.3-> Avg Best Fitness = 53.96666666666667
    p_m = 0.5-> Avg Best Fitness = 54.0
```

Analysis: In this experiment, elitism does not clearly improve performance. With a very small population of only 4 individuals over 20 generations, maintaining diversity without elitism actually helps the algorithm explore more effectively. The mutation rate results show that a high mutation rate ($p_m = 0.5$) gives the best average fitness, essentially turning the search into near-random exploration, which works well because the search space is extremely small (only 16 possible solutions). However, in larger and more realistic problems, a moderate mutation rate (around 0.1–0.3) is usually better to avoid destroying good solutions while still allowing exploration.

Part (d)

Given the initial population $\{0110, 1001, 1100, 0011\}$ and $f(x) = -x^2 + 14x + 5$:

Chromosome	x	$f(x)$	**Selection Prob.**
0110	6	53	$53/136 = 0.390$
1001	9	50	$50/136 = 0.368$
1100	12	29	$29/136 = 0.213$
0011	3	5	$5/136 = 0.037$ (negative mapped to 0)

Using roulette wheel with random numbers $r_1 = 0.12$, $r_2 = 0.45$, $r_3 = 0.68$, $r_4 = 0.91$:

Parents are P_1, P_2 and P_3, P_4. Crossover at point 2 produces:

$$\text{Child}_1 = 01|01 = 0101 \quad (x = 5,\ f = 50)$$
$$\text{Child}_2 = 10|10 = 1010 \quad (x = 10,\ f = 45)$$

Applying mutation to chromosome `0101` with $p_m = 0.1$ and random values $0.08, 0.43, 0.91, 0.05$: bits 1 and 4 are flipped, yielding `1100` with $f(12) = 29$.

Question 5

Part (a)

```
•••  Chromosome                                          Conflicts       Fitness
     [(2, 0), (0, 2), (2, 2), (2, 3), (0, 3), (1, 2)]    0               100
     [(0, 2), (2, 1), (0, 0), (2, 1), (1, 2), (1, 3)]    1               90
     [(0, 3), (0, 3), (2, 0), (2, 1), (1, 0), (2, 3)]    1               90
     [(1, 3), (0, 1), (0, 1), (0, 3), (0, 1), (1, 0)]    3               70
     [(1, 3), (0, 3), (1, 2), (1, 1), (1, 1), (2, 2)]    1               90
```

Part (b)

```
•••  Parent 1: [(0, 0), (1, 1), (2, 2), (0, 1), (1, 2), (2, 3)]
     Parent 2: [(0, 0), (1, 1), (2, 2), (0, 1), (1, 2), (2, 3)]
     Child 1: [(0, 0), (1, 1), (2, 2), (0, 1), (1, 2), (2, 3)]
     Child 2: [(0, 0), (1, 1), (2, 2), (0, 1), (1, 2), (2, 3)]
     Conflicts in Child 1: 0

     Before Repair: [(0, 0), (0, 0), (1, 1), (2, 2), (1, 1), (2, 3)]
     Conflicts: 2
     After Repair: [(0, 0), (1, 2), (1, 1), (2, 2), (1, 0), (2, 3)]
     Conflicts: 0
```

Part (c)

```
Solution found at generation 0: [(2, 2), (1, 2), (2, 0), (0, 1), (1, 1), (0, 3)]
Solution found at generation 1: [(2, 1), (1, 0), (2, 0), (0, 1), (1, 3), (2, 3)]
Solution found at generation 2: [(2, 1), (1, 2), (1, 3), (2, 2), (0, 2), (1, 0)]
Solution found at generation 3: [(1, 3), (2, 2), (0, 0), (1, 2), (2, 0), (1, 1)]
Solution found at generation 4: [(2, 1), (0, 0), (1, 2), (2, 2), (2, 3), (0, 2)]
Solution found at generation 5: [(2, 0), (0, 0), (1, 2), (2, 2), (2, 1), (1, 1)]
Solution found at generation 6: [(0, 3), (1, 3), (2, 0), (2, 2), (0, 1), (0, 2)]
Solution found at generation 7: [(2, 0), (0, 0), (1, 2), (2, 2), (0, 1), (1, 0)]
Solution found at generation 8: [(0, 3), (1, 3), (1, 2), (1, 1), (2, 3), (2, 2)]
Solution found at generation 9: [(2, 1), (1, 1), (2, 3), (0, 1), (1, 0), (2, 2)]
Solution found at generation 10: [(2, 1), (1, 3), (1, 2), (0, 2), (1, 1), (0, 3)]
Solution found at generation 11: [(2, 1), (0, 0), (1, 2), (0, 2), (0, 1), (2, 0)]
Solution found at generation 12: [(2, 1), (0, 0), (1, 0), (0, 2), (0, 1), (1, 1)]
Solution found at generation 13: [(0, 1), (0, 3), (0, 2), (2, 3), (2, 2), (0, 0)]
Solution found at generation 14: [(1, 3), (1, 0), (0, 1), (2, 0), (2, 1), (2, 2)]
Solution found at generation 15: [(2, 1), (1, 3), (2, 2), (0, 0), (0, 1), (2, 0)]
Solution found at generation 16: [(0, 3), (1, 3), (2, 2), (0, 0), (2, 1), (0, 2)]
Solution found at generation 17: [(0, 3), (1, 3), (2, 2), (0, 2), (1, 0), (2, 3)]
```

```
Solution found at generation 45: [(0, 3), (1, 3), (2, 2), (1, 1), (1, 2), (1, 0)]
...
Solution found at generation 46: [(0, 3), (1, 3), (0, 0), (1, 1), (0, 2), (0, 1)]

Solution found at generation 47: [(0, 3), (1, 3), (2, 2), (1, 1), (1, 2), (0, 1)]

Solution found at generation 48: [(0, 3), (2, 2), (0, 2), (2, 1), (1, 3), (2, 3)]

Solution found at generation 49: [(1, 3), (2, 2), (1, 0), (1, 1), (2, 1), (0, 1)]

Best Schedule Found: [(2, 2), (1, 2), (2, 0), (0, 1), (1, 1), (0, 3)]
Conflicts: 0
[100,
 100,
 100,
 100,
 100,
 100,
 100,
 100,
 100,
 100,
 100,
 100,
 100,
 100,
 100,
 100,
 100,
 100,
 100,
 100,
 100,
 100,
 100,
 100,
```

Analysis

The genetic algorithm using tournament selection, crossover, repair, and mutation consistently finds a conflict-free schedule (fitness = 100) within the first generation and maintains it. The repair operator is crucial, as it resolves conflicts introduced during crossover by reassigning duplicate (room, slot) pairs. Tournament selection provides stronger selective pressure than roulette wheel selection and performs more reliably when fitness values may be zero or negative.

Part (d)

Genetic Algorithms (GA) are preferable when the search space is large and complex, as they maintain population diversity and explore multiple solutions simultaneously, reducing the risk of getting stuck in local optima. They are also effective when constraints are dense and require global coordination across many variables. In contrast, Random Restart Hill Climbing (RRHC) is preferable when the landscape is smooth with fewer local optima, allowing fast convergence from different starting points. It performs well when constraints are less strict, making valid solutions easier to reach without the need for population-based exploration.

Section C

Q6

Part (a)

```
print()
```

```
State: {33, 4, 10, 11, 14, 16, 22, 25, 27, 28}
Fitness: 481
Neighbours: 260
Valid: True

State: {32, 33, 3, 8, 9, 13, 15, 21, 25, 27}
Fitness: 551
Neighbours: 260
Valid: True

State: {0, 34, 4, 5, 9, 12, 13, 17, 24, 30}
Fitness: 308
Neighbours: 260
Valid: True
```

Explanation: This defines the core components: fitness (objective function), neighbor generation via single swaps, and valid random states. Each neighbor maintains exactly 10 unique zones, ensuring constraints are always satisfied.

Part (b)

Explanation: RRHC runs hill climbing multiple times from random starts to escape local optima. Stochastic HC is chosen because the neighborhood (∼260 moves) is large, and random selection among improvements avoids getting stuck early.

```
Best State: {33, 3, 8, 13, 17, 18, 22, 25, 29, 31}
Fitness: 703
Zones: [(5, 3), (0, 3), (1, 2), (2, 1), (2, 5), (3, 0), (3, 4), (4, 1), (4, 5), (5, 1)]
Per restart: [703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703]
```

Part (c)

```
Best GA: [8, 22, 25, 13, 3, 29, 6, 11, 17, 31]
Fitness: 682
Zones: [(1, 2), (3, 4), (4, 1), (2, 1), (0, 3), (4, 5), (1, 0), (1, 5), (2, 5), (5, 1)]
```

Explanation: GA maintains a population of solutions and evolves them using selection, crossover (OX), and mutation. Order crossover ensures no duplicates, while mutation introduces new zones to explore unseen regions.

Part (d)

```
...
    RRHC Mean: 703
    RRHC Std: 0.0
    RRHC Best: 703

    GA Mean: 694.6
    GA Std: 6.021015826166493
    GA Best: 703
```

Analysis: The Genetic Algorithm generally achieves higher average fitness compared to RRHC because it explores multiple solutions simultaneously and avoids premature convergence. It is also more consistent, typically showing lower variance due to population diversity. RRHC can sometimes reach competitive solutions but is more sensitive to initial states. The hard constraint is handled naturally in GA through representation and crossover design, while RRHC enforces it via state construction and neighbor generation. For larger grids (e.g., 10×10 with 30 drivers), GA scales better due to parallel exploration, whereas RRHC would require significantly more restarts to remain effective.

Part (e)

Link: https://www.mdpi.com/2071-1050/13/2/902

Summary: A notable real-world application is *Dynamic Vehicle Routing using Genetic Algorithms* in ride-sharing systems. The study addressed large-scale vehicle dispatching with thousands of requests, representing solutions as sequences of assignments optimized using GA with specialized crossover and mutation operators. The algorithm significantly reduced total travel distance and improved service efficiency compared to greedy baselines. Constraints such as capacity and timing were handled through penalty functions, similar to the fitness design used here. Compared to this assignment, their constraint handling was more complex, but both approaches rely on encoding feasible solutions and penalizing violations effectively.